

4주차_5장: CCN

5.3 전이 학습(199-241)

합성곱 신경망 기반 딥러닝 모델 훈련을 위해서는 큰 데이터셋이 필요함

but, 큰 데이터셋 확보 어려움 → 전이학습(transfer learning)

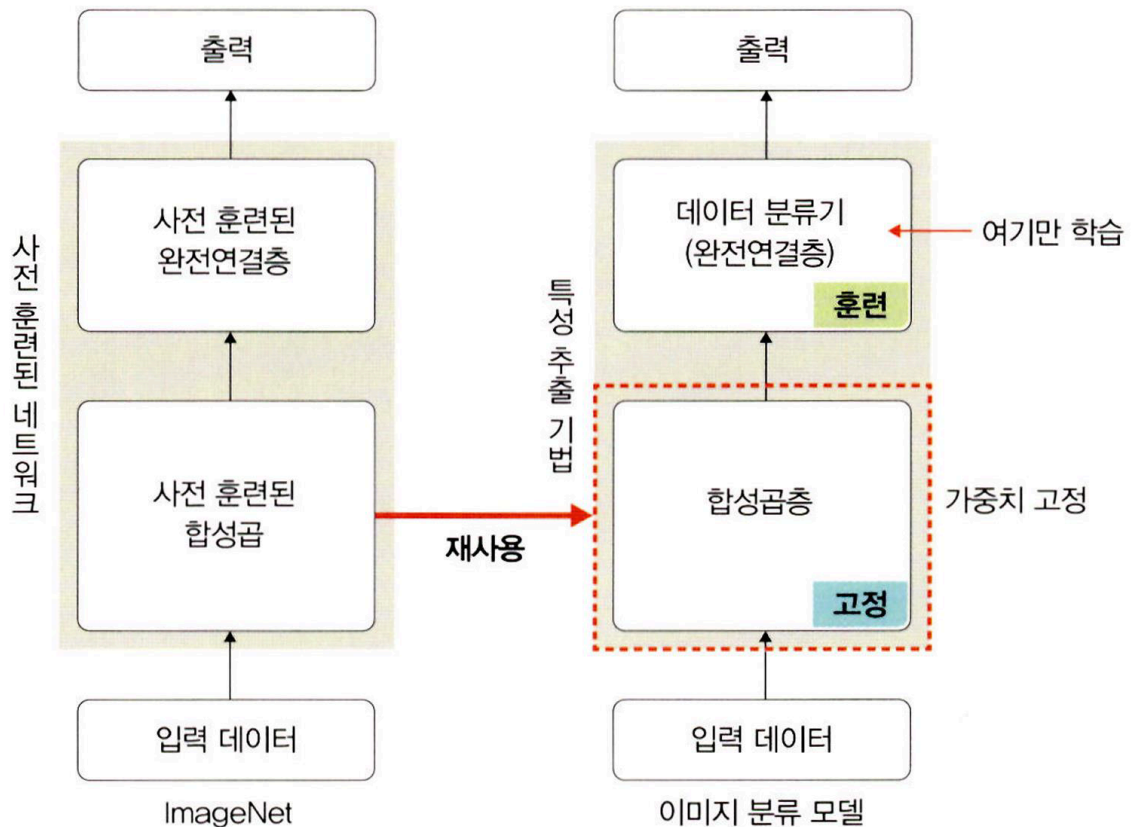
- 이미지넷(ImageNet)과 같이 큰 데이터셋을 써 훈련된 모델의 가중치를 가져와 해결하려는 과제에 맞게 보정해 사용
- 아주 큰 데이터셋을 사용해 훈련된 모델: 사전 훈련된 모델(네트워크)
- 전이학습

문제 A → (해결) 문제 A를 해결하며 얻은 지식, 경험 → (적용) 문제 B

5.3.1 특성 추출 기법(199-243)

- 특성 추출(feature extractor): ImageNet 데이터셋으로 사전 훈련된 모델을 가져온 후 마지막에 완전연결층 부분만 새로 만듦
- 학습할 때 마지막 완전연결층(이미지의 카테고리를 결정하는 부분)만 학습; 나머지 계층은 학습되지 않도록 함
- 특성 추출의 두 부분
 - 합성곱층: 합성곱층과 풀링층
 - 데이터 분류기(완전연결층): 추출된 특성을 입력받아 최종적으로 이미지에 대한 클래스 분류
- 사용되는 이미지 분류 모델: Xception, Inception V3, ResNet50...

▼ 그림 5-30 특성 추출 기법



5-12: 라이브러리 호출

```
import os
import times
import copy
import glob
import cv2
import shutil

import torch
import torchvision
import torchvision.transforms as transforms
import torchvision.models as models
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader

import matplotlib.pyplot as plt
```

```
from google.colab import drive
drive.mount('/content/drive')

data_path = '/content/drive/MyDrive/euron/4주차/080289-main/chap05/data/catanddog/train'
```

5-13: 이미지 데이터 전처리 방법 정의

```
transform = transforms.Compose([
    transforms.Resize([256, 256]),
    transforms.RandomResizedCrop(224),
    transforms.RandomHorizontalFlip(),
    transforms.ToTensor()
])
train_dataset = torchvision.datasets.ImageFolder(
    data_path,
    transform=transform
)
train_loader = torch.utils.data.DataLoader(
    train_dataset,
    batch_size = 32,
    num_workers = 8,
    shuffle=True
)

print(len(train_dataset))
```

- torchvision.transform: 이미지 데이터를 변환하여 모델(네트워크)의 입력으로 사용할 수 있게 변환해 해줌
 - Resize: 이미지 크기 조정; 합성곱층 통과를 위해 이미지 크기 조정하는 전처리 과정
 - RandomResizedCrop: 이미지 랜덤한 크기 및 비율로 자름; 데이터 확장 용도로 사용
 - RandomHorizontalFlip: 이미지를 랜덤하게 수평으로 뒤집음
 - ToTensor: 이미지를 텐서로 변환
- datasets.ImageFolder: 데이터로더가 데이터를 불러올 대상과 방법 정의
 - train_dataset = torchvision.datasets.ImageFolder(data_path, transform=transform)
 - data_path: 불러올 데이터가 위치한 경로
 - transform: 이미지 데이터에 대한 전처리
- DataLoader: ImageFolder(train_dataset)을 데이터로더에 할당; 불러올 데이터양 결정하는 batch_size 지정
 - train_loader = torch.utils.data.DataLoader(train_dataset, batch_size=32, num_workers=8, shuffle=True)
 - train_dataset: 데이터셋 지정
 - batch_size: 한 번에 불러올 데이터양 결정하는 배치 크기 설정
 - num_workers: 데이터 불러올 때 몇 개의 하위의 프로세스 사용할지
 - shuffle: 데이터 무작위로 섞을지

RandomResizedCrop

-

```
samples, labels = next(iter(train_loader))
classes = {0: 'cat', 1: 'dog'}
fig = plt.figure(figsize=(16, 24))
for i in range(24):
    a = fig.add_subplot(4, 6, i+1)
```

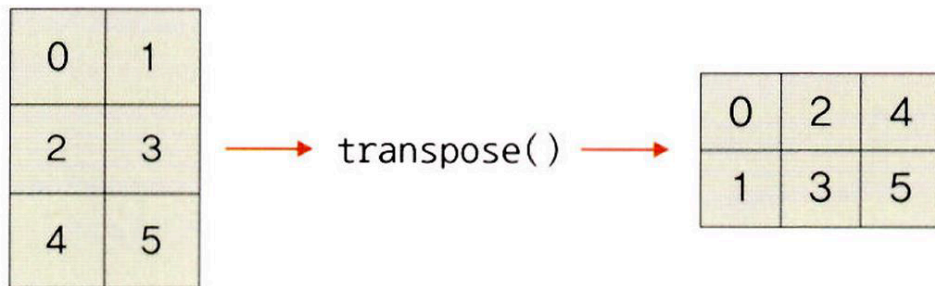
```

a.set_title(classes[labels[i].item()])
a.axis('off')
a.imshow(np.transpose(samples[i].numpy(), (1, 2, 0)))
plt.subplots_adjust(bottom=0.2, top=0.6, hspace=0)

```

- 학습에 사용될 이미지 출력
 - 반복자(iterator, for 구문과 같은 효과) 사용 위해 iter(), next() 필요
 - iter(): 전달된 데이터의 반복자를 꺼내 반환
 - next(): 반복자가 다음에 출력해야 할 요소 반환
 - iter()로 반복자 구하고, next()에 전달하여 차례대로 꺼냄
 - train_loader에서 데이터 하나 씩 꺼냄
 - np.transpose - 행과 열을 바꾸어 행렬의 차원을 바꿈

▼ 그림 5-33 transpose()



- 행렬의 내적 연산을 가능하게 만들기 위해 np.transpose()나 np.reshape() 등으로 차원 조정 필요

▼ 그림 5-34 행렬의 내적

곱셈, 덧셈

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} \begin{bmatrix} b_{11} & b_{12} & b_{13} \\ b_{21} & b_{22} & b_{23} \\ b_{31} & b_{32} & b_{33} \end{bmatrix} = \begin{bmatrix} c_{11} & c_{12} & c_{13} \\ c_{21} & c_{22} & c_{23} \\ c_{31} & c_{32} & c_{33} \end{bmatrix}$$

A B C

$$\begin{bmatrix} c_{11} \\ c_{21} \\ c_{31} \end{bmatrix} = \begin{bmatrix} a_{11}b_{11} + a_{12}b_{21} + a_{13}b_{31} \\ a_{21}b_{11} + a_{22}b_{21} + a_{23}b_{31} \\ a_{31}b_{11} + a_{32}b_{21} + a_{33}b_{31} \end{bmatrix}$$

```
resnet18 = models.resnet18(pretrained=True)
```

ResNet18

- 50개의 계층으로 구성된 합성곱 신경망
- ImageNet 데이터베이스 100만개 넘는 영상 이용해 훈련된 신경망
- 입력 제약이 큼
- 충분한 메모리(RAM) 없으면 학습 속도 느림

사전 훈련된 모델

- 파이토치에서 무작위 가중치로 모델 구성하기

```
import torchvision.models as models
resnet18= models.resnet18()
alexnet = models.alexnet()
vgg16 = models.vgg16()
squeezenet = models.squeezenet1_0()
desenet = models.densenet161()
inception = models.inception_v3()
googlenet = models.googlenet()
shufflenet = models.shufflenet_v2_x1_0()
```

```
mobilenet_v3_small = models.mobilenet_v3_small()
resnext50_32x4d = models.resnext50_32x4d()
wide_resnet50_2 = models.wide_resnet50_2()
mnasnet = models.mnasnet1_0()
```

- 사전 학습된 모델(사전 학습된 모델의 가중치 값) 사용

```
import torchvision.models as models
resnet18 = models.resnet18(pretrained=True)
alexnet = models.alexnet(pretrained=True)
squeezenet = models.squeezenet1_0(pretrained=True)
vgg16 = models.vgg16(pretrained=True)
desenet = models.densenet161(pretrained=True)
inception = models.inception_v3(pretrained=True)
googlenet = models.googlenet(pretrained=True)
shufflenet = models.shufflenet_v2_x1_0(pretrained=True)
mobilenet_v3_small = models.mobilenet_v3_small(pretrained=True)
resnext50_32x4d = models.resnext50_32x4d(pretrained=True)
wide_resnet50_2 = models.wide_resnet50_2(pretrained=True)
mnasnet = models.mnasnet1_0(pretrained=True)
```

```
def set_parameter_requires_grad(model, feature_extracting=True):
    if feature_extracting:
        for param in model.parameters():
            param.requires_grad = False

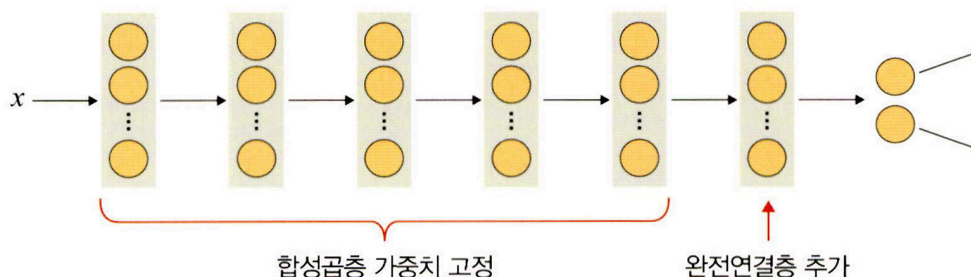
set_parameter_requires_grad(resnet18)
```

- 사전 훈련된 모델의 파라미터 학습 유무 지정
 - ResNet18의 합성곱층을 사용하되 파라미터에 대해서는 학습하지 않도록 함
 - param.requires_grad = False: 역전파 중 파라미터들에 대한 변화 계산할 필요 없음 나타냄
 - 모델의 일부를 고정하고 나머지 학습하고자 할 때 requires_grad = False로 설정
 - 모델의 일부 - 합성곱층(convolutional layer), 풀링(pooling)층

```
resnet18.fc = nn.Linear(512, 2)
```

- ResNet18에 완전연결층 추가

♥ 그림 5-36 ResNet18에 완전연결층 추가



- nn.Linear(512, 2): 2는 클래스가 두 개라는 뜻

```
for name, param in resnet18.named_parameters():
    if param.requires_grad:
        print(name, param.data)
```

- 모델 파라미터 값들 확인
 - model.named_parameters(): 모델에 접근해 파라미터 값들 가져올 때 사용

```
model = models.resnet18(pretrained=True)

for param in model.parameters():
    param.requires_grad = False

model.fc = torch.nn.Linear(512, 2)
for param in model.fc.parameters():
    param.requires_grad = True

optimizer = torch.optim.Adam(model.fc.parameters())
cost = torch.nn.CrossEntropyLoss()
print(model)
```

- 모델 객체 생성, 손실 함수 정의
 - models.resnet18(pretrained=True): 모델의 객체 생성
 - for param in model.parameters(): param.requires_grad = False 모델의 합성곱층 가중치 고정
 - for param in model.fc.parameters(): param.requires_grad=True: 완전연결층은 학습
 - cost = torch.nn.CrossEntropyLoss(): 손실함수 정의

```
def train_model(model, dataloaders, criterion, optimizer, device, num_epochs=13, is_train=True):
    since = time.time()
    acc_history = []
    loss_history = []
    best_acc = 0.0

    for epoch in range(num_epochs):
        print('Epoch {}/{}'.format(epoch, num_epochs-1))
        print('-'*10)

        running_loss = 0.0
        running_corrects = 0

        for inputs, labels in dataloaders:
            inputs = inputs.to(device)
            labels = labels.to(device)

            model.to(device)
            optimizer.zero_grad()
            outputs = model(inputs)
            loss = criterion(outputs, labels)
            _, preds = torch.max(outputs, 1)
            loss.backward()
            optimizer.step()
```

```

        running_loss += loss.item() * inputs.size(0)
        running_corrects += torch.sum(preds == labels.data)

    epoch_loss = running_loss / len(dataloaders.dataset)
    epoch_acc = running_corrects.double() / len(dataloaders.dataset)

    print('Loss {:.4f} Acc: {:.4f}'.format(epoch_loss, epoch_acc))

    if epoch_acc > best_acc:
        best_acc = epoch_acc

    acc_history.append(epoch_acc.item())
    loss_history.append(epoch_loss)
    torch.save(model.state_dict(), os.path.join('/content/drive/MyDrive/euron/4주차/080289-main/chap05/data/catanddog', '{0:0=2d}.pth'.format(epoch)))

    print()

    time_elapsed = time.time() - since
    print('Training complete in {:.0f}m {:.0f}s'.format(time_elapsed // 60, time_elapsed % 60))
    print('Best Acc: {:.4f}'.format(best_acc))
    return acc_history, loss_history

```

- 모델 학습을 위한 함수 생성

- since = time.time(): 컴퓨터의 현재 시각 구하는 함수
- for epoch in range(num_epochs): 앞서 num_epochs 13으로 정의; 에포크(13)만큼 반복
- for inputs, labels in dataloaders: 데이터 로더에 전달된 데이터만큼 반복
- optimizer.zero_grad(): 기울기 0으로 설정
- outputs = model(inputs): 순전파 학습
- loss.backward(): 역전파 학습
- running_loss += loss.item() * inputs.size(0): 출력 결과와 레이블을 오차를 계산한 결과 누적해 저장
- running_corrects += torch.sum(preds == labels.data): 출력 결과와 레이블이 동일한지 확인한 결과를 누적해 저장
- epoch_loss = running_loss / len(dataloaders.dataset): 평균 오차 저장
- epoch_acc = running_corrects.double() / len(dataloaders.dataset): 평균 정확도 계산
- torch.save(model.state_dict(), os.path.join('/content/drive/MyDrive/euron/4주차/080289-main/chap05/data/catanddog', '{0:0=2d}.pth'.format(epoch)))
: 모델 재사용을 위해 저장
- return acc_history, loss_history: 모델의 정확도와 오차 반환

```

params_to_update = []

for name, param in resnet18.named_parameters():
    if param.requires_grad == True:
        params_to_update.append(param)
    print("\t", name)

optimizer = optim.Adam(params_to_update)

```

- 파라미터 학습 결과 옵티마이저에 전달
 - `params_to_update.append(param)`: 파라미터 학습 결과 저장
 - `optimizer = optim.Adam(params_to_update)`: 학습 결과 옵티마이저에 전달

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
criterion = nn.CrossEntropyLoss()
train_acc_hist, train_lost_hist = train_model(resnet18, train_loader, criterion, optimizer, device)
```

- 모델 학습
 - `criterion= nn.CrossEntropyLoss`: 손실함수 지정

```
from torch._dynamo.convert_frame import Tracker
test_path = '/content/drive/MyDrive/euron/4주차/080289-main/chap05/data/catanddog/test'

transform = transforms.Compose(
    [
        transforms.Resize(224),
        transforms.CenterCrop(224),
        transforms.ToTensor(),
    ]
)
test_dataset = torchvision.datasets.ImageFolder(
    root=test_path,
    transform=transform
)
test_loader = torch.utils.data.DataLoader(
    test_dataset,
    batch_size = 32,
    num_workers =1,
    shuffle =True
)

print(len(test_dataset))
```

- 테스트 데이터 호출 및 전처리

```
def eval_model(model, dataloaders, device):
    since = time.time()
    acc_history = []
    best_acc = 0.0

    model_dir = '/content/drive/MyDrive/euron/4주차/080289-main/chap05/data/catanddog'
    saved_models = glob.glob(os.path.join(model_dir, '*.pth'))
    saved_models.sort()
    print('saved_model', saved_models)

    for model_path in saved_models:
        print('Loading model', model_path)

        model.load_state_dict(torch.load(model_path))
        model.eval()
        model.to(device)
```

```

    running_corrects = 0

    for inputs, labels in dataloaders:
        inputs = inputs.to(device)
        labels = labels.to(device)

        with torch.no_grad():
            outputs = model(inputs)

            _, preds = torch.max(outputs.data, 1)
            preds[preds >= 0.5] = 1
            preds[preds < 0.5] = 0
            running_corrects += preds.eq(labels.cpu()).int().sum()

    epoch_acc = running_corrects.double() / len(dataloaders.dataset)
    print('Acc: {:.4f}'.format(epoch_acc))

    if epoch_acc > best_acc:
        best_acc = epoch_acc
    acc_history.append(epoch_acc.item())
    print()

    time_elapsed = time.time() - since
    print('Validation complete in {:.0f}m {:.0f}s'.format(time_elapsed // 60, time_elapsed % 60))
    print('Best Acc: {:.4f}'.format(best_acc))

    return acc_history

```

- 테스트 데이터 평가 함수 생성
 - saved_models.sort(): 불러온 .pth 파일들 정렬
 - for inputs, labels in dataloaders: 테스트 반복
 - with torch.no_grad(): autograd 사용 X
 - outputs = model(inputs): 데이터를 모델에 적용한 결과를 outputs에 저장
 - saved_models = glob.glob(os.path.join(model_dir, '*.pth'))
 - 현재 디렉토리에서 원하는 파일만 추출해 가져올 때 사용
 - ../catanddog/ 경로에서 pth 확장자를 갖는 파일을 가져오라는 의미
 - _, preds = torch.max(outputs.data, 1)
 - torch.max: 주어진 텐서 배열의 최댓값이 들어있는 index 반환하는 함수
 - ex. y_pred = [[0.2, 0.7, 0.8, 0.4]]라는 배열에 torch.max(y_pred.data, 1) 적용하면 최댓값 0.8, 인덱스 2 반환
 - running_corrects += preds.eq(labels.cpu()).int().sum()
 - preds 배열과 labels 일치하는지 검사하는 용도로 사용
 - .sum(): 모델의 예측 결과와 정답(레이블)이 일치하는 것들의 개수 합을 숫자로 출력

```
val_acc_hist = eval_model(resnet18, test_loader, device)
```

- 테스트 데이터를 평가 함수에 적용

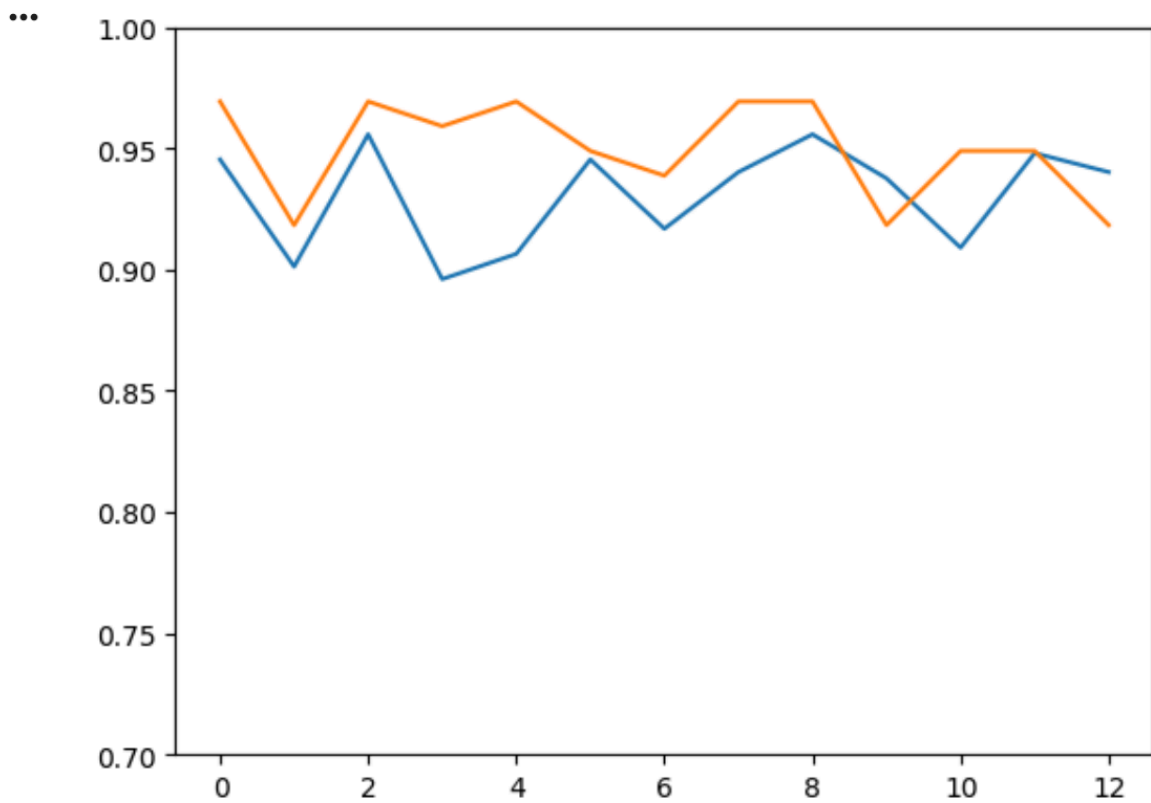
```

plt.plot(train_acc_hist)
plt.plot(val_acc_hist)

```

```
plt.show()
```

- 훈련과 테스트 데이터 정확도 그래프로 확인



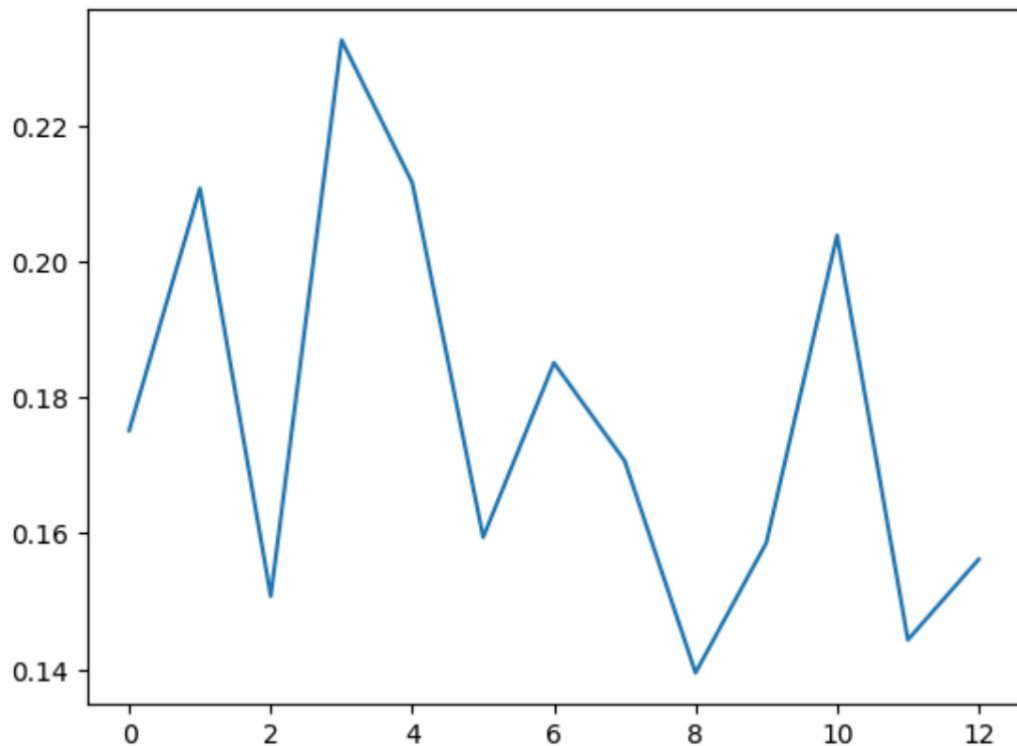
```
plt.plot(train_acc_hist)
plt.show()
```

- 훈련 데이터의 오차에 대한 그래프 확인



plt.show()

...



```
def im_convert(tensor):
    image = tensor.clone().detach().numpy()
    image = image.transpose(1, 2, 0)
    image = image * (np.array((0.5, 0.5, 0.5)) + np.array((0.5, 0.5, 0.5)))
    image = image.clip(0, 1)
    return image
```

- 예측 이미지 출력을 위한 전처리 함수
 - tensor.clone(): 기존 텐서의 내용을 복사한 텐서 생성하겠다는 의미
 - detach(): 기존 텐서에서 기울기가 전파되지 않는 텐서
 - tensor.clone().detach(): 기존 텐서를 복사한 새로운 텐서를 생성하지만, 기울기에 영향을 주지 않음

▼ 표 5-2 tensor.clone(), tensor.detach(), tensor.clone().detach()의 비교

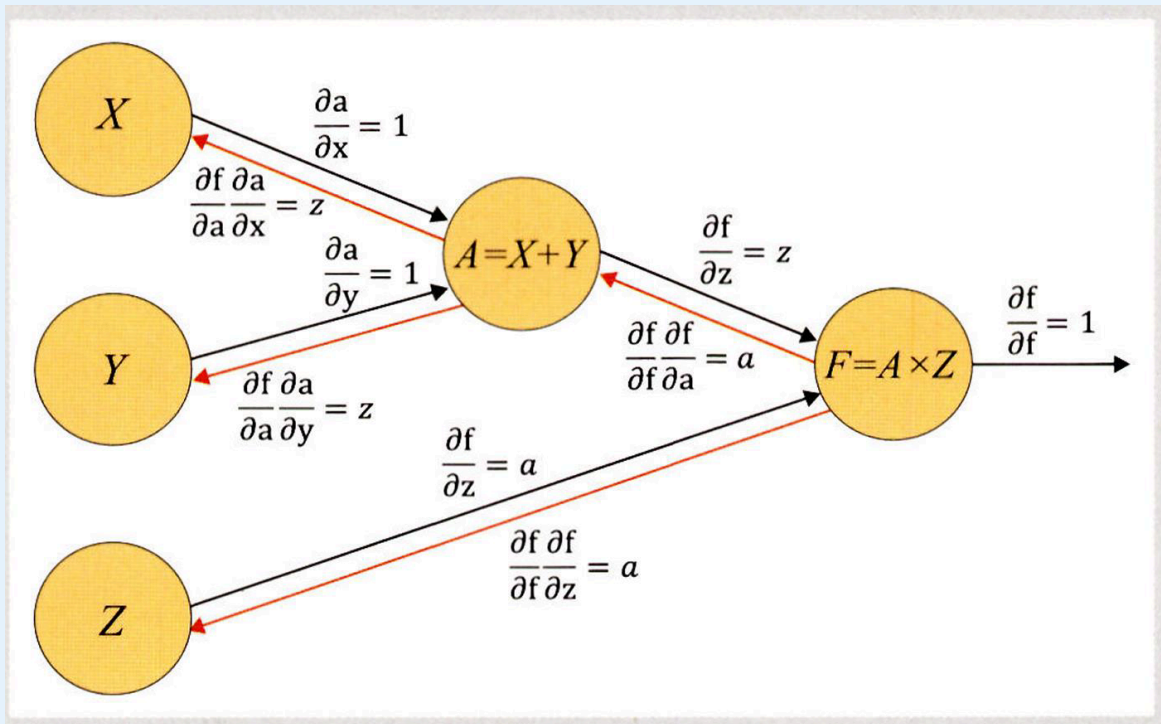
구분	메모리	계산 그래프 상주 유무
tensor.clone()	새롭게 할당	계산 그래프에 계속 상주
tensor.detach()	공유해서 사용	계산 그래프에 상주하지 않음
tensor.clone().detach()	새롭게 할당	계산 그래프에 상주하지 않음

- `image.clip(0, 1)`: 입력 값이 주어진 범위를 벗어날 때 입력 값을 특정 범위로 제한시키기 위해 사용; image 데이터를 0과 1 사이의 값으로 제한

```
import numpy as np
exam = np.array([-1.8, -1.2, -0.7, 0.0, 1.4, 1.9])
print(exam)
print(np.clip(exam, -0.5, 0.5))
```

계산 그래프(computational graph)

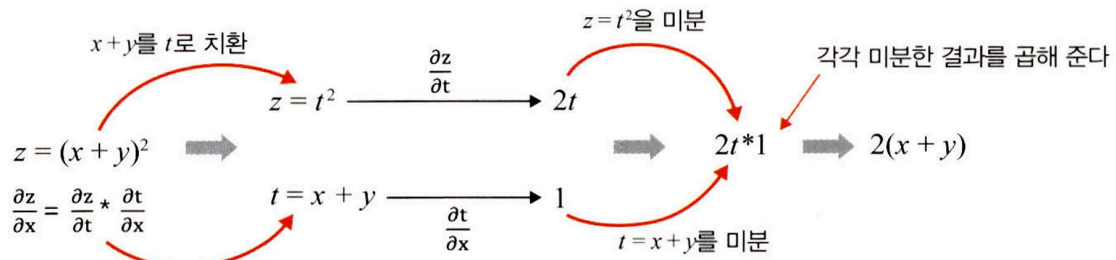
- 계산 과정을 그래프로 나타냄
- 여러 개의 node와 node를 연결하는 edge로 구성됨



- 국소적 계산 가능
- 역전파를 통한 미분 계산 편리- 연쇄법칙(chain rule)을 이용해 빠르고 간편하게 미분 계산 가능

연쇄 법칙(chain rule)

- 두 개 이상의 함수가 결합된 함수
- 합성 함수의 미분법



```
classes = {0: 'cat', 1: 'dog'}

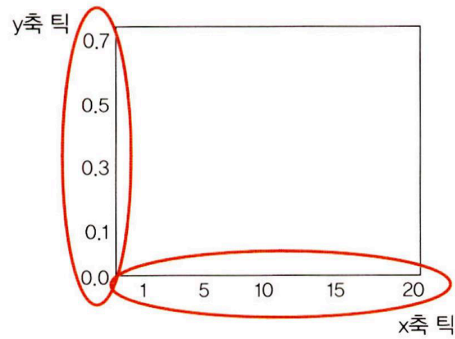
dataiter = iter(test_loader)
images, labels = next(dataiter)
output = model(images)
_, preds = torch.max(output, 1)

fig = plt.figure(figsize=(25, 4))
for idx in np.arange(20):
    ax = fig.add_subplot(2, 10, idx+1, xticks=[], yticks=[])
    plt.imshow(im_convert(images[idx]))
    ax.set_title("{}({})".format(str(classes[preds[idx].item()]), str(classes[labels[idx].item()]))), color = ("green" if preds[idx]==labels[idx] else "red"))

plt.show()
plt.subplots_adjust(bottom=0.2, top=0.6, hspace=0)
```

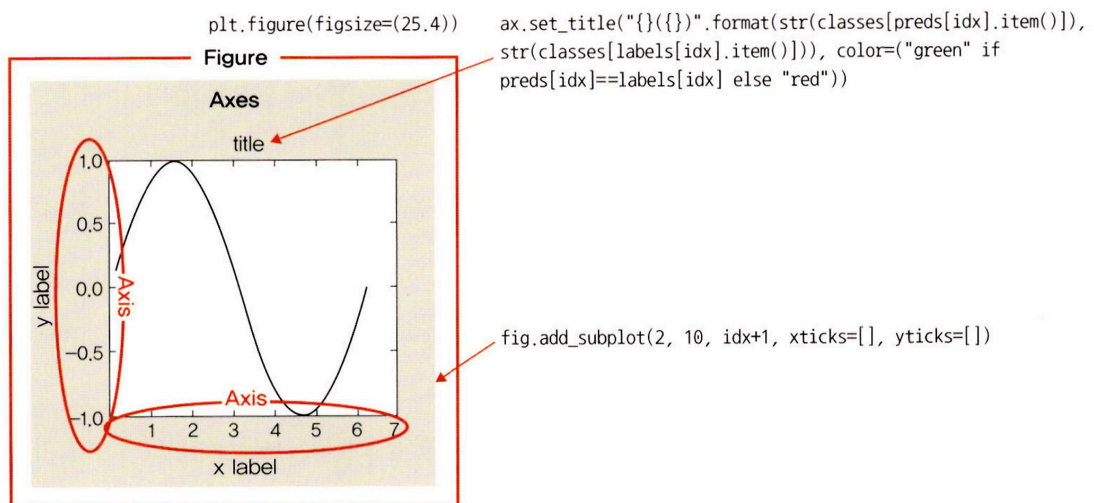
- 개와 고양이 예측 결과 출력
 - `images, labels = next(dataiter)`: 테스트 데이터셋에서 이미지와 레이블을 분리해 가져옴
 - `ax = fig.add_subplot(2, 10, idx+1, xticks=[], yticks=[])`
 - `add_subplot` 한 화면에 여러 개의 이미지 담기 위해 사용
 - `add_subplot(2, 10, idx+1, xticks=[], yticks=[])`
 - 2: 행의 수, 이미지 두 줄로 출력
 - 10: 열의 수, 한 줄에 10개의 이미지 출력
 - `idx+1`: 인덱스 의미, 행과 열 기준으로 순차적으로 이미지 출력
 - `xticks=[], yticks=[]`: tick을 삭제하겠다는 의미

♥ 그림 5-41 x축, y축 틱



- `ax.set_title("{}({})".format(str(classes[preds[idx].item()]), str(classes[labels[idx].item()])), color = ("green" if preds[idx]==labels[idx] else "red"))`
 - `classes[preds[idx].item()]`: `preds[idx].item()` 값이 `classes`로 정의된 0과 1 중 어떤 값을 갖는지 판별하겠다는 의미
- `plt.subplots_adjust(bottom=0.2, top=0.6, hspace=0)`
 - figure 안 subplot(그림 같은 경우에는 axes)의 위치를 조정할 때 사용

♥ 그림 5-42 서브플롯



예측결과

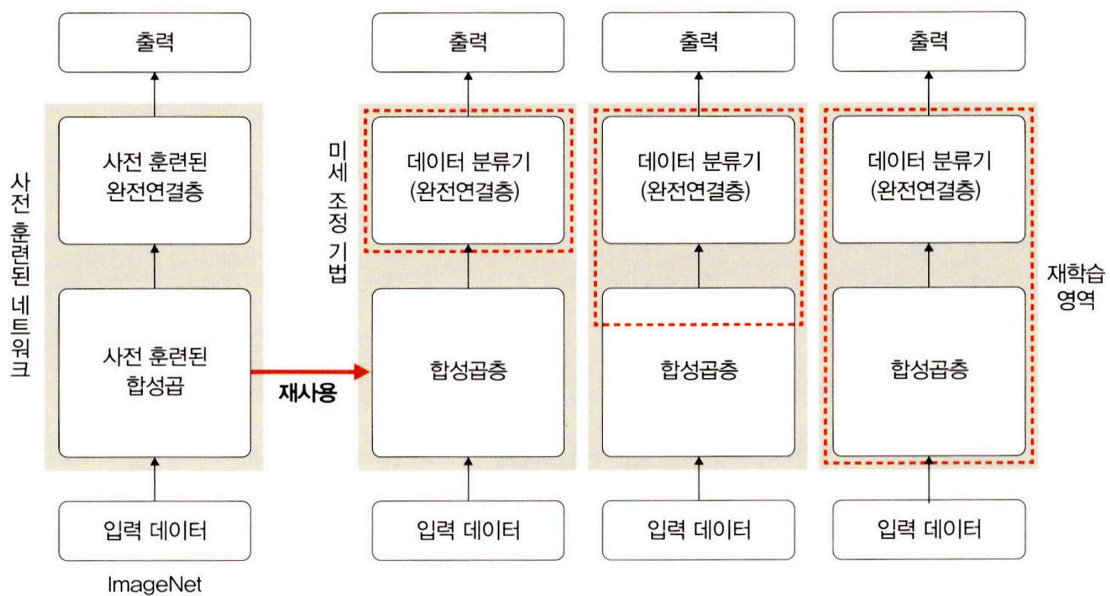


5.3.2 미세 조정 기법(fine-tuning)

- 사전 훈련된 모델과 합성곱층, 데이터 분류기의 가중치를 업데이트 하여 훈련시키는 방식
- 특성 추출

- 목표 특성을 잘 추출했다는 전제 하 좋은 성능을 낼 수 있음
- 특성 추출이 잘못된 경우 - ex. ImageNet 데이터셋의 이미지 특징과 전자상거래 물품의 이미지 특징이 다름 → 미세 조정 기법으로 새로운(전자상거래) 이미지 데이터를 사용해 네트워크 가중치 업데이트해 특성을 다시 추출할 수 있음
- 사전 학습된 모델을 목적에 맞게 재학습시키거나 학습된 가중치의 일부를 재학습 시킴
- 미세 조정 기법 전략
 - 데이터셋이 크고 사전 훈련된 모델과 유사성이 작을 경우: 모델 전체 재학습. 데이터셋의 크기가 크기 때문에 재학습이 좋음
 - 데이터셋이 크고 사전 훈련된 모델과 유사성이 클 경우: 합성곱층의 뒷부분(완전연결층과 가까운 부분)과 데이터 분류기 학습. 데이터셋이 유사하기 때문에 전체를 학습시키기보다 강한 특징이 나타나는 합성곱층의 뒷부분과 데이터 분류기만 새로 학습해도 됨
 - 데이터셋이 작고 사전 훈련된 모델과 유사성이 작을 경우: 합성곱층 일부분과 데이터분류기 학습; 데이터셋이 작기 때문에 일부 계층에 미세 조정 기법을 적용해도 효과가 없을 수 있음
 - 데이터셋이 작고 사전 훈련된 모델과 유사성이 클 경우: 데이터분류기만 학습. 데이터가 적기 때문에 많은 계층에 미세 조정 기법을 적용하면 과적합 발생할 수 있음; 최종 데이터 분류기인 완전 연결층에서만 미세 조정 기법 적용

▼ 그림 5-44 미세 조정 기법



5.4 설명 가능한 CNN

- 설명 가능한 CNN(explainable CNN): 딥러닝 처리 결과를 사람이 이해할 수 있는 방식으로 제시하는 기술
 - CNN은 블랙박스과 같아 내부에서 어떻게 동작하는지 설명하기 어려움
 - → CNN 처리 과정을 시각화해야함

▼ 그림 5-45 CNN의 블랙박스



- CNN을 구성하는 각 중간 계층부터 최종 분류까지 입력된 이미지에서 특성이 어떻게 추출되고 학습하는지 시각적으로 설명할 수 있어야 함

5.4.1 특성 맵 시각화

- 특성 맵(feature map)은 입력 이미지 또는 다른 특성 맵처럼 필터를 입력에 적용한 결과
- 특성 맵에서 입력 특성을 감지하는 방법을 이해할 수 있도록 돕는 것

```
import matplotlib.pyplot as plt
from PIL import Image
import cv2
import torch
import torch.nn.functional as F
import torch.nn as nn
from torchvision.transforms import ToTensor
import torchvision
import torchvision.transforms as transforms
import torchvision.models as models

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

- 필요한 라이브러리 호출

```
from torch.nn.modules.pooling import MaxPool2d
class XAI(torch.nn.Module):
    def __init__(self, num_classes=2):
        super(XAI, self).__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 64, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(64),
            nn.ReLU(inplace=True), #기존의 데이터를 연산의 결과값으로 대체
            nn.Dropout(0.3),
            nn.Conv2d(64, 64, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(64),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=2, stride=2),

            nn.Conv2d(64, 128, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(128),
            nn.ReLU(inplace=True),
            nn.Dropout(0.4),
            nn.Conv2d(128, 128, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(128),
```

```

        nn.ReLU(inplace=True),
        nn.MaxPool2d(kernel_size=2, stride=2),

        nn.Conv2d(128, 256, kernel_size=3, padding=1, bias=False),
        nn.BatchNorm2d(256),
        nn.ReLU(inplace=True),
        nn.Dropout(0.4),
        nn.Conv2d(256, 256, kernel_size=3, padding=1, bias=False),
        nn.BatchNorm2d(256),
        nn.ReLU(inplace=True),
        nn.Dropout(0.4),
        nn.Conv2d(256, 256, kernel_size=3, padding=1, bias=False),
        nn.BatchNorm2d(256),
        nn.ReLU(inplace=True),
        nn.MaxPool2d(kernel_size=2, stride=2),

        nn.Conv2d(256, 512, kernel_size=3, padding=1, bias=False),
        nn.BatchNorm2d(512),
        nn.ReLU(inplace=True),
        nn.Dropout(0.4),
        nn.Conv2d(512, 512, kernel_size=3, padding=1, bias=False),
        nn.BatchNorm2d(512),
        nn.ReLU(inplace=True),
        nn.Dropout(0.4),
        nn.Conv2d(512, 512, kernel_size=3, padding=1, bias=False),
        nn.BatchNorm2d(512),
        nn.ReLU(inplace=True),
        nn.MaxPool2d(kernel_size=2, stride=2),

        nn.Conv2d(512, 512, kernel_size=3, padding=1, bias=False),
        nn.BatchNorm2d(512),
        nn.ReLU(inplace=True),
        nn.Dropout(0.4),
        nn.Conv2d(512, 512, kernel_size=3, padding=1, bias=False),
        nn.BatchNorm2d(512),
        nn.ReLU(inplace=True),
        nn.Dropout(0.4),
        nn.Conv2d(512, 512, kernel_size=3, padding=1, bias=False),
        nn.BatchNorm2d(512),
        nn.ReLU(inplace=True),
        nn.MaxPool2d(kernel_size=2, stride=2),
    )
    self.classifier = nn.Sequential(
        nn.Linear(512, 512, bias=False),
        nn.Dropout(0.5),
        nn.BatchNorm1d(512),
        nn.ReLU(inplace=True),
        nn.Dropout(0.5),
        nn.Linear(512, num_classes)
    )

    def forward(self, x):
        x = self.features(x)
        x = x.view(x.size(0), -1)

```

```
x = self.classifier(x)
return F.log_softmax(x)
```

설명 가능한 네트워크 생성

- 13개의 합성곱층과 두 개의 완전연결층으로 구성된 네트워크 생성
- 합성곱층과 완전연결층 레루 사용
- return F.log_softmax(x): 신경망 말단 결괏값들을 확률 개념으로 해석하기 위해 softmax 함수 결과에 log 취함
- softmax의 기울기 소멸 문제 때문에 log softmax 사/tntl

$$\text{softmax}(x)_i = \frac{e^{x_i}}{\sum_{j=1} e^{x_j}}$$

$$\text{logsoftmax} = \log\left(\frac{e^{x_i}}{\sum_{j=1} e^{x_j}}\right)$$

$$= x_i - \log\left(\sum_{j=1} e^{x_j}\right)$$

```
model = XAI() #model이라는 이름의 객체 생성
model.to(device) #model을 장치에 할당
model.eval() # 테스트 데이터에 대한 모델 평가 용도로 사용
```

- 모델 객체화

```
class LayerActivations:
    features = []
    def __init__(self, model, layer_num):
        self.hook = model[layer_num].register_forward_hook(self.hook_fn)

    def hook_fn(self, module, input, output):
        self.features = output.detach().cpu().numpy()

    def remove(self):
        self.hook.remove()
```

특성 맵을 확인하기 위한 클래스 정의

- 특성 맵: 합성곱층을 입력 이미지와 필터를 연산하여 얻은 결과
- self.hook = model[layer_num].register_forward_hook(self.hook_fn)
 - 매 계층 print 문을 사용하지 않더라도 hook 기능을 사용해 각 계층의 활성화 함수 및 기울기 값을 확인할 수 있음
 - register_forward_hook: 순전파 중 각 네트워크 모듈의 입력 및 출력 가져오는 것

```
import torch
x = torch.Tensor([0,1,2,3]).requires_grad_()
y = torch.Tensor([4,5,6,7]).requires_grad_()
w = torch.Tensor([1,2,3,4]).requires_grad_()
z=x+y
o = w.matmul(z)
o.backward()
print(x.grad, y.grad, z.grad, w.grad, o.grad)
```

- 코드에서 o와 z는 특정한 값으로 정의되지 않은 중간변수

- 파이토치는 이런 변수에 대해 기울기 값을 저장하지 않음
- 중간변수에 대해 `z.register_hook(hook_fn)` 이용하면 기울기 값 알 수 있음

```
img = cv2.imread("/content/drive/MyDrive/euron/4주차/080289-main/chap05/data/cat.jpg")
plt.imshow(img)
img = cv2.resize(img, (32, 32), interpolation=cv2.INTER_LINEAR)
img = ToTensor()(img).unsqueeze(0)
print(img.shape)
```

이미지 호출

- `img = cv2.resize(img, (100, 100), interpolation=cv2.INTER_LINEAR)`: `cv2.resize` 이미지 크기 변경할 때 사용
 - `img`: 변경할 이미지 파일
 - `(32, 32)`: 변경될 이미지 크기를 (너비, 높이)로 지정
 - `interpolation`: 보간법
 - 이미지 크기 변경할 경우 변형된 이미지의 픽셀 추정해 값을 할당해야함
 - 이미지 비율을 변경할 때 존재하지 않는 영역에 새로운 픽셀 값을 매핑하거나 존재하는 픽셀들을 압축해 새로운 값을 할당해야함
 - 이 상황을 피하기 위해 이미지 상에 존재하는 픽셀 데이터 (x_i, y_i) 들에 대한 근사 함수 $f(x, y)$ 를 적용해 새로운 픽셀 값을 구함
- `img = ToTensor()(img).unsqueeze(0)`: `unsqueeze()` 1차원 데이터 생성하는 함수
 - 이미지 텐서로 변환, 변환된 데이터를 1차원으로 변경

```
import torch

x1 = torch.rand(3, 10, 64)
x2 = x1.unsqueeze(dim=0)
print(x2.shape)
print('-----')
x3 = x1.unsqueeze(dim=1)
print(x3.shape)
```

결과) `torch.Size([1, 3, 10, 64])`
`torch.Size([3, 1, 10, 64])`

```
result = LayerActivations(model.features, 0)

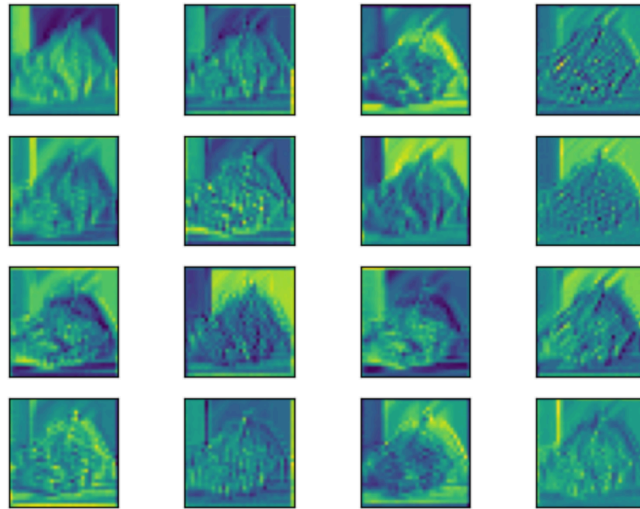
model(img.to(device))
activations = result.features
```

- Conv2d 특성 맵 확인

```
fig, axes = plt.subplots(4, 4)
fig = plt.figure(figsize=(12,8))
fig.subplots_adjust(left=0, right=1, bottom=0, top=1, hspace=0.05, wspace=0.05)
for row in range(4):
    for column in range(4):
        axis = axes[row][column]
        axis.get_xaxis().set_ticks([])
        axis.get_yaxis().set_ticks([])
        axis.imshow(activations[0][row*10+column])

plt.show()
```

- 특성 맵 시각적으로 표현
- 첫번째 Conv2d 계층에서 특성 맵에 대한 출력 결과
- 입력층과 가까운 계층; 입력 이미지의 형태 많이 유지됨



<Figure size 1200x800 with 0 Axes>

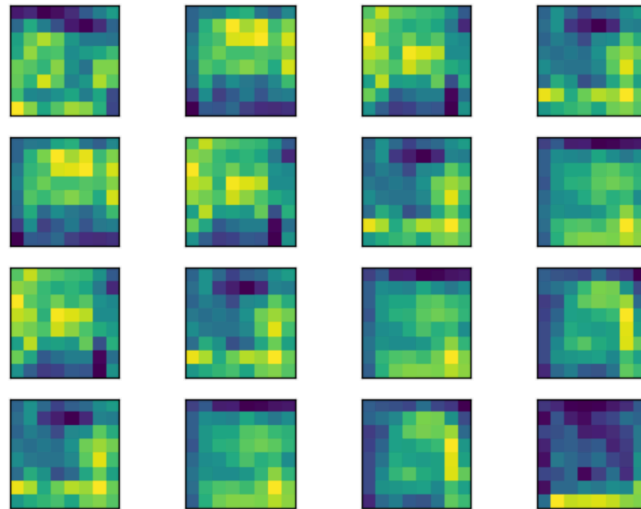
```
result = LayerActivations(model.features, 20)

model(img)
activations = result.features
```

- 20번째 계층에 대한 특성 맵

```
fig, axes = plt.subplots(4, 4)
fig = plt.figure(figsize=(12, 8))
fig.subplots_adjust(left=0, right=1, bottom=0, top=1, hspace=0.05, wspace=0.05)
for row in range(4):
    for column in range(4):
        axis = axes[row][column]
        axis.get_xaxis().set_ticks([])
        axis.get_yaxis().set_ticks([])
        axis.imshow(activations[0][row*1+column])

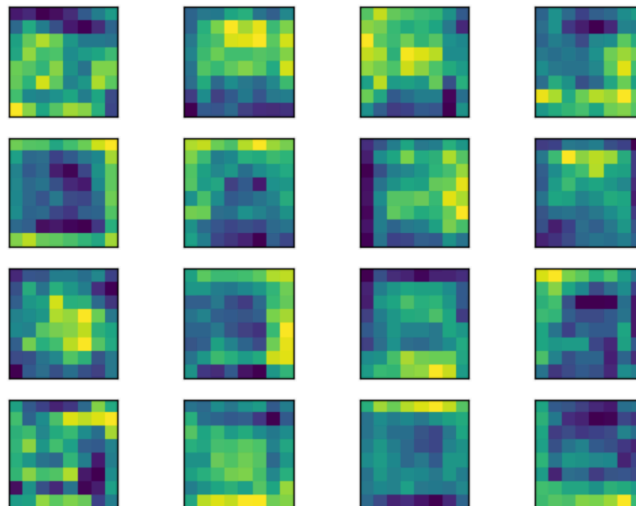
plt.show()
```



<Figure size 1200x800 with 0 Axes>

```
fig, axes = plt.subplots(4, 4)
fig = plt.figure(figsize=(12, 8))
fig.subplots_adjust(left=0, right=1, bottom=0, top=1, hspace=0.05, wspace=0.05)
for row in range(4):
    for column in range(4):
        axis = axes[row][column]
        axis.get_xaxis().set_ticks([])
        axis.get_yaxis().set_ticks([])
        axis.imshow(activations[0][row*10+column])

plt.show()
```



<Figure size 1200x800 with 0 Axes>

- 출력층에 가까울수록 원래 형태 찾아볼 수 없음
- 이미지 특징들만 전달됨

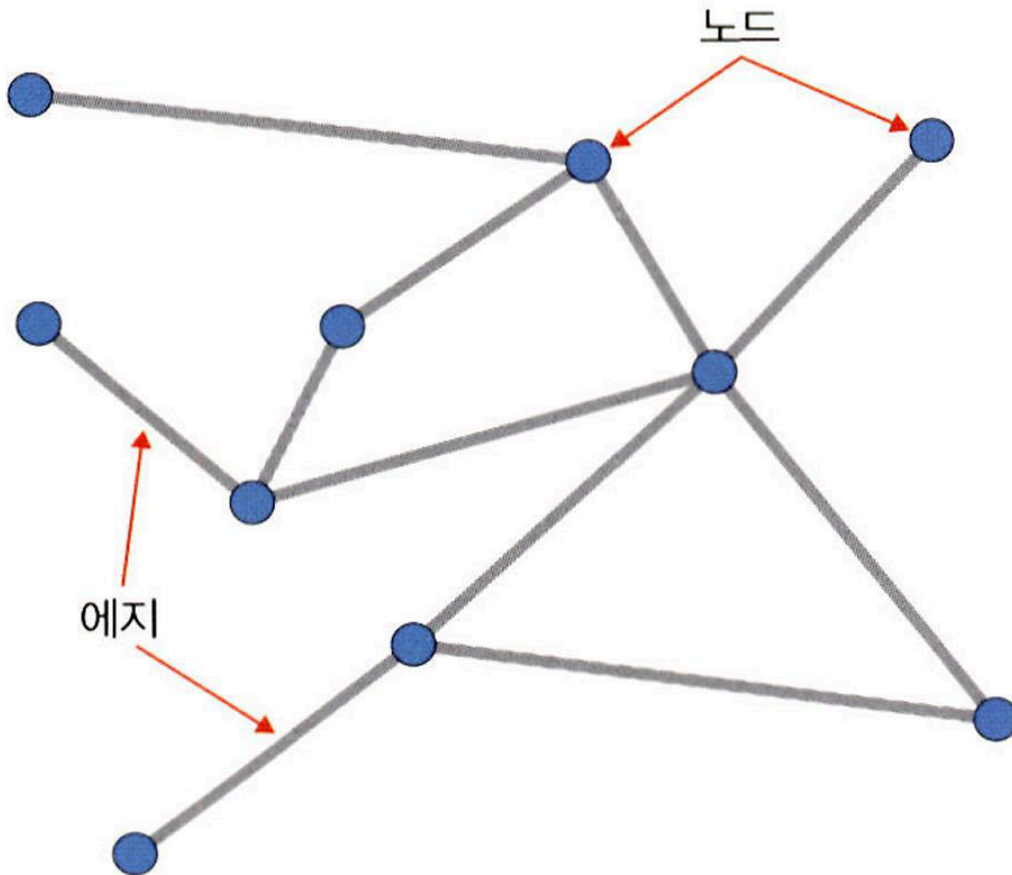
5.5 그래프 합성곱 네트워크

- 그래프 합성곱 네트워크(graph convolution network): 그래프 데이터를 위한 신경망

5.5.1 그래프란

- 방향성이 있거나(directed) 없는(undirected) edge로 연결된 nodes(=vertices)의 집합

▼ 그림 5-50 그래프

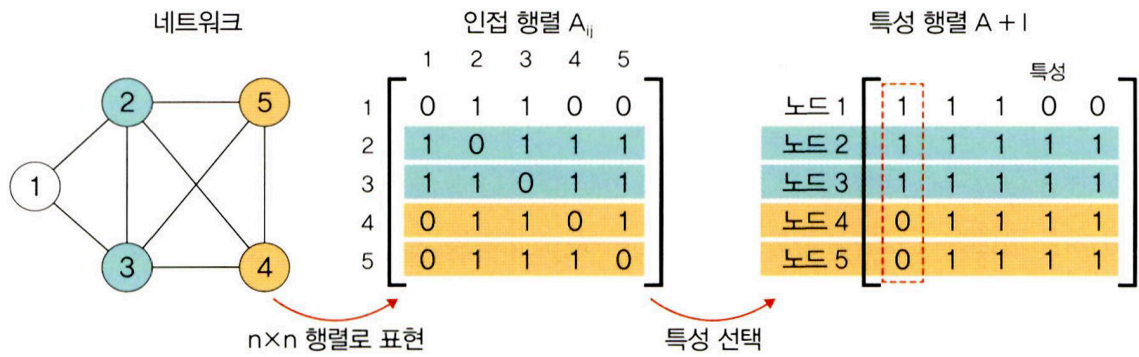


- 그래프의 구성 요소
 - 노드: 파란색 원
 - 에지: 두 노드를 연결한 선

5.5.2 그래프 신경망

- 그래프 신경망(graph Neural Network, GNN): 그래프 구조에서 사용하는 신경망
- 그래프 데이터에 대한 표현
 1. 인접 행렬(adjacency matrix)
 - a. 노드 n 개를 $n \times n$ 행렬(matrix)로 표현
 - b. 생성된 인접 행렬 내 값은 A_{ij} 는 i 와 j 의 관련성 여부를 만족하는 값으로 채워 줌
 2. 특성 행렬(feature matrix)
 - a. 인접 행렬만으로는 특성을 파악하기 어려움 ; 단위 행렬 적용
 - b. 각 입력 데이터에서 이용할 특성 선택
 - c. 특성 행렬에서 각 행은 선택된 특성에 대해 각 노드가 갖는 값

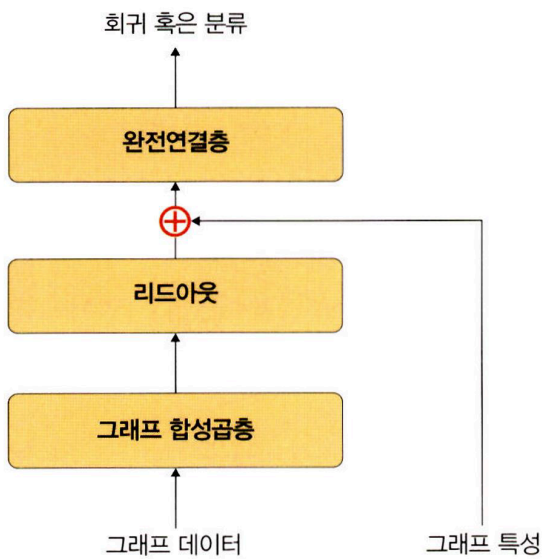
▼ 그림 5-51 특성 행렬⁵



5.5.3 그래프 합성곱 네트워크

- 그래프 합성곱 네트워크(Graph Convolutional Network, GCN): 이미지에 대한 합성곱을 그래프 데이터로 확장한 알고리즘

▼ 그림 5-52 그래프 합성곱 네트워크



- 리드아웃(readout): 특성 행렬을 하나의 벡터로 변환하는 함수
 - 전체 노드의 특성 벡터에 대해 평균을 구하고 그래프 전체를 표현하는 하나의 벡터 생성
- 합성곱층(graph convolutional layer): 그래프 합성곱층을 이용한 그래프 형태의 데이터는 행렬 형태의 데이터로 변환; 딥러닝 알고리즘 적용할 수 있음
- GCN의 활용
 - SNS에서 관계 네트워크
 - 학술 연구에서 인용 네트워크
 - 3D mesh